

ASSIGNMENT

- 2

CSE - 221

Submitted to : Ariyan Sir

Submitted by : Sheikh Md. Tahmid Hossain

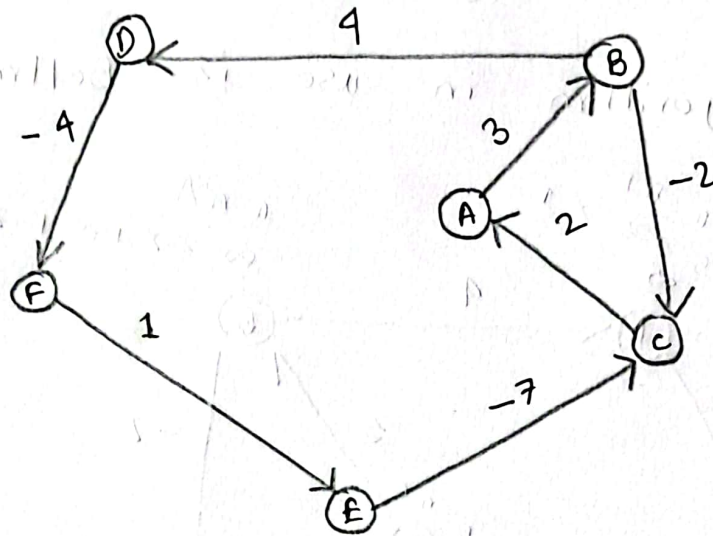
ID : 22299522

Section ; 24

Summer '24

Date : 25 / 9 / 24

Answer to the Question No. 1



(a)

The algorithm which we learned to find out the shortest distance is known as Dijkstra's Algorithm. An essential condition was its weight would be non-negative. Calculating for the above graph:-

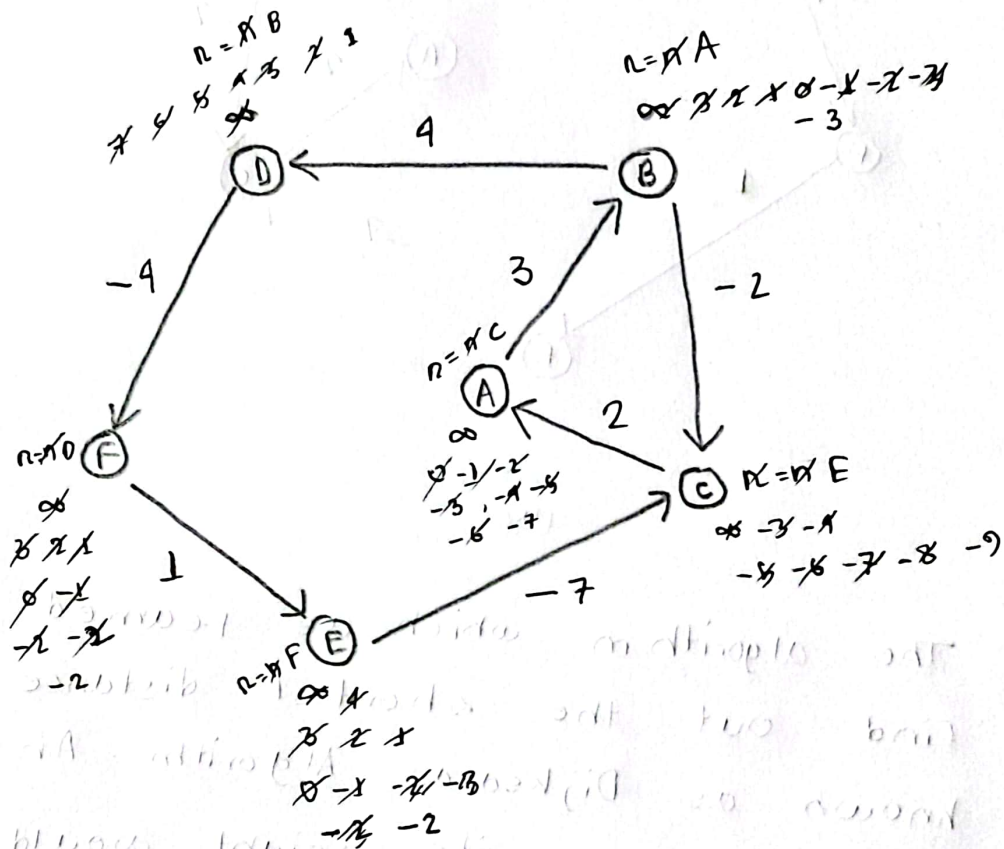
$$(3 + 4 + 1 - 4 - 7 + 2 = -1)$$

So, it is not possible to determine the shortest path as it forms

a negative weight

(b)

The algorithm in use is Bellman-ford



Given source is, A.

Number of vertices = 7

So, we need to iterate it $(V-1)$
= 6 times

So, the mechanism for detecting a negative weighted cycle is if we iterate more than 6 times and find any updates, it means negative weight cycle exists.

So, the edges are:-

(A-B), (B-D), (D-F), (F-E), (E-C),
(C-A), (B-C)

	A	B	D	F	E	C
i = 0	0	∞	∞	∞	∞	∞
i = 1	-1	3	7	3	4	-3
i = 2	-2	2	6	2	3	-4
i = 3	-3	1	5	1	2	-5
i = 4	-4	0	4	0	1	-6
i = 5	-5	-1	3	-1	0	-7
i = 6	-6	-2	2	-2	-1	-8
i = 7	-7	-3	1	-3	-2	-9

We notice that if we conduct an extra iteration value updates, so, negative weighted cycle exists.

(c)

The required algorithm would be Dijkstra's algorithm. It has a linear time complexity which is $O((V+E) \log V)$.

(d)

We are to use a modified version of Dijkstra's algorithm.

We need to do these in order:-

- (i) Consider each node as source after completion of a node.

Source $\rightarrow$ Each Node (Loop Over)

for each $s \in V[G]$

$$d[s] = 0$$

$$Q = V[G]$$

MODIFIED DIJKSTRA (G, s)

for each $v \in V[G]$

do $d[v] \rightarrow \infty$

$\pi[v] \rightarrow n$

for each $s \in V[G]$

$d[s] = 0$

$Q = V[G]$

} change

while $Q \neq \emptyset$

do $u \rightarrow \text{Extract-MIN}(Q)$

for each vertex $v \in \text{Adj}[u]$

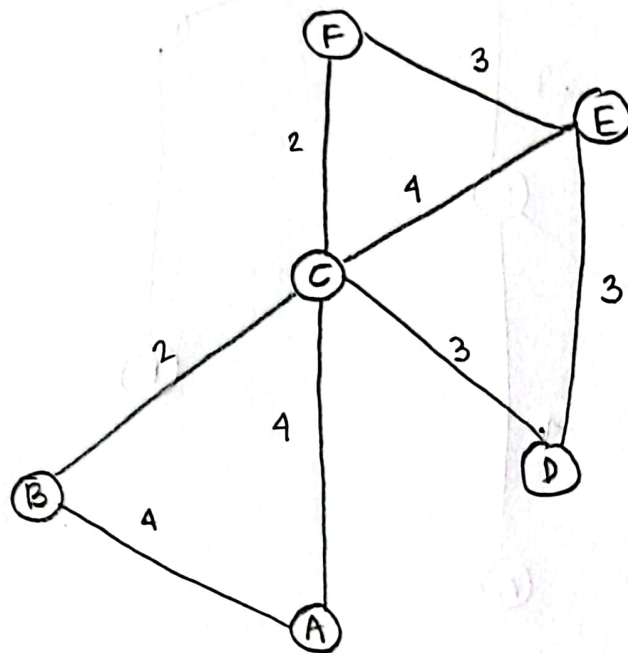
do if $d[v] > d[u] + w(u, v)$

then $d[v] \rightarrow d[u] + w(u, v)$

$\pi[v] \rightarrow u$

Answer to the Question No.2

(a)



The algorithm applied is Kruskal's

Algorithm.

$$(B-A) \rightarrow 4 \text{ X}$$

$$(A-D) \rightarrow 4 \text{ X}$$

$$(A-C) \rightarrow \textcircled{4}$$

$$(C-B) \rightarrow \textcircled{2}$$

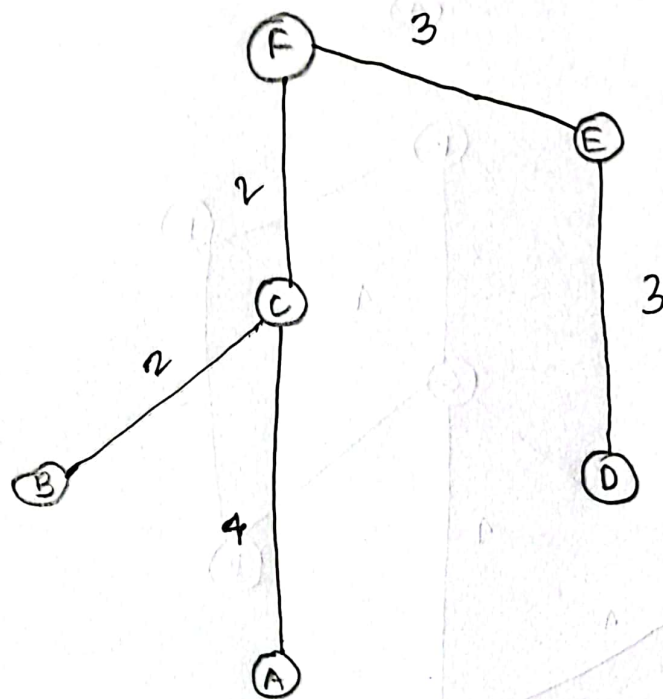
$$(C-D) \rightarrow 3 \text{ X}$$

$$(D-E) \rightarrow \textcircled{3}$$

$$(E-C) \rightarrow 4 \text{ X}$$

$$(C-F) \rightarrow \textcircled{2}$$

$$(F-E) \rightarrow \textcircled{3}$$

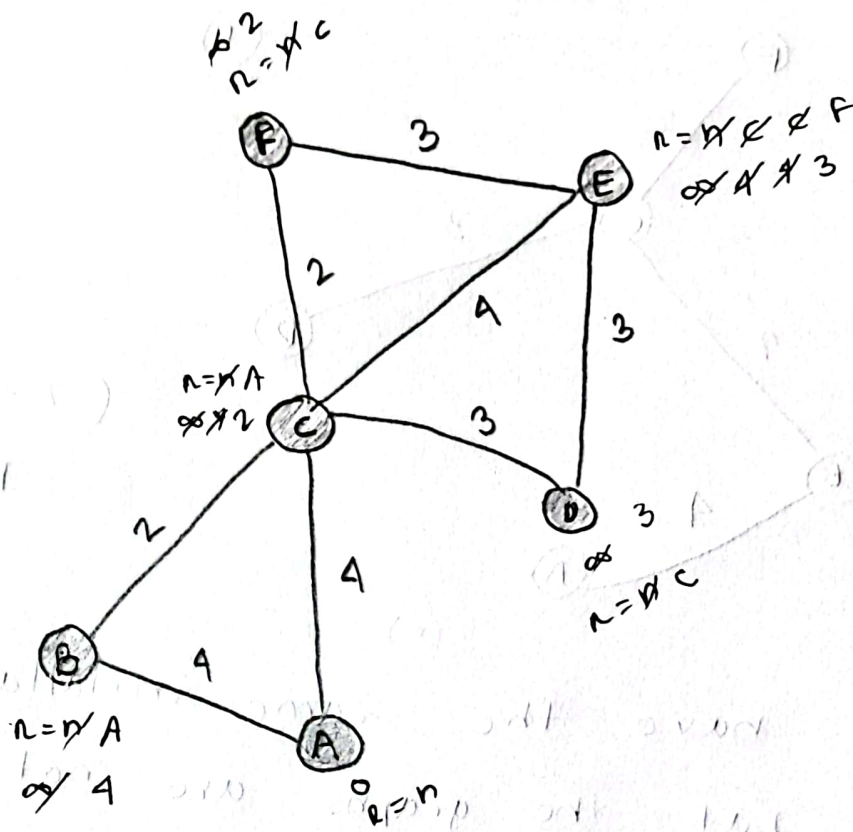


$$\rightarrow 2 + 2 + 3 + 3 + 4 = 14$$

So, the total resulting installation cost will be 14. The applied algorithm is Kruskal's Algo.

(c)

The other algorithm that could be used is Prim's Algorithm



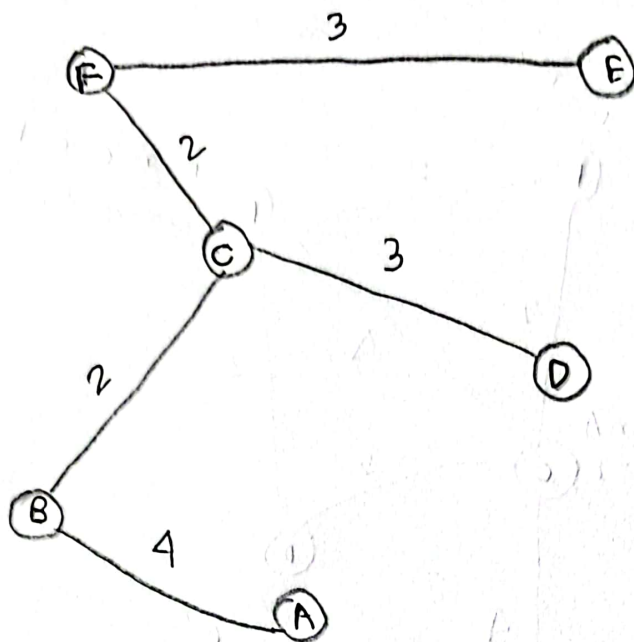
Priority Queue, Q

$Q = [A, B, C, D, E, F]$

$u = [A, B, C, D, E]$

$v = [B, C, D, A, E, F, D, C, E, E]$

So, the graph has been drawn below:-



$$(2 + 2 + 3 + 3 + 4)$$

$$= \cancel{16} \quad 14$$

(c)

Both have the same installation cost but the graphs are not same. The edges differ in both graphs. Thus modifying them will result in similar graphs.

~~The graphs whose~~ ed.

The graphs could be same if there was direction, given so the edges had a particular order.

MW (G)

$p = \{ \}$

for each edge $(u, v) \in [G]$

if $w(u, v) \in D$:

$D[w(u, v)] += 1$

$new_w = w(u, v) + D[w(u, v)]$

else:

$D[w(u, v)] = 0$

return G

Answer to the Question No. 3

(a)

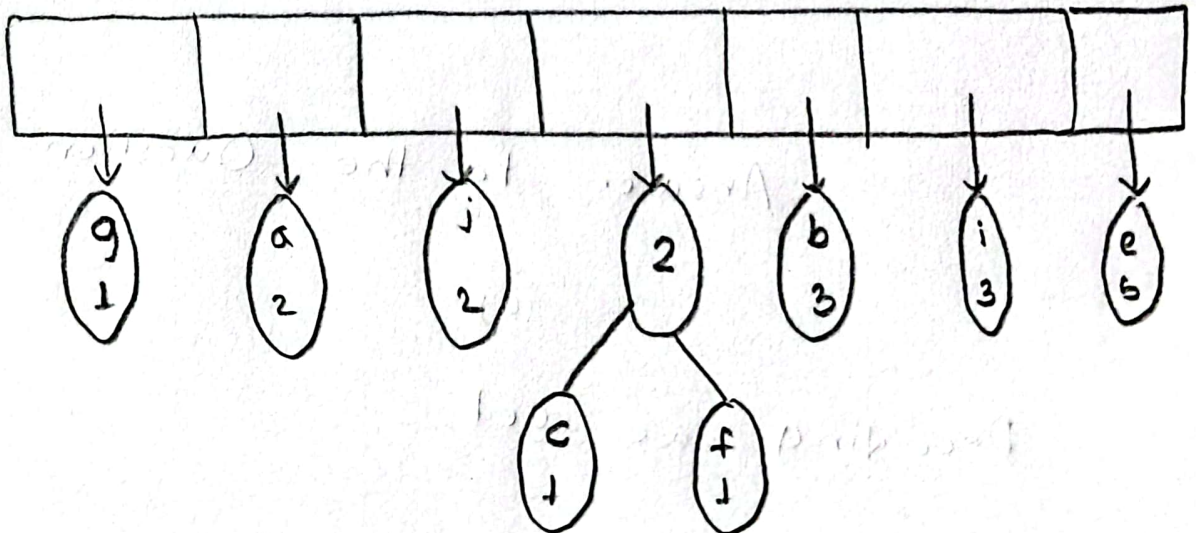
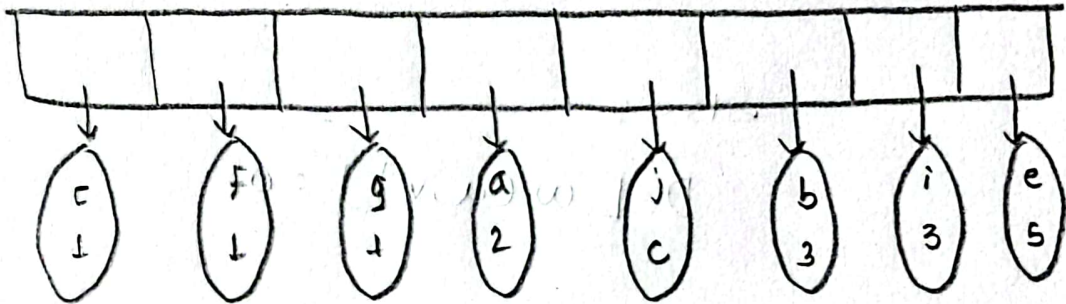
Decoding we get,

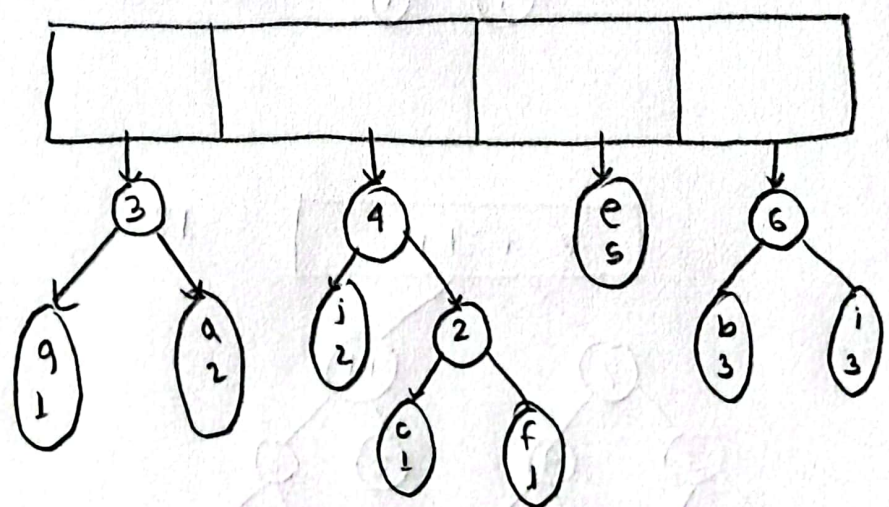
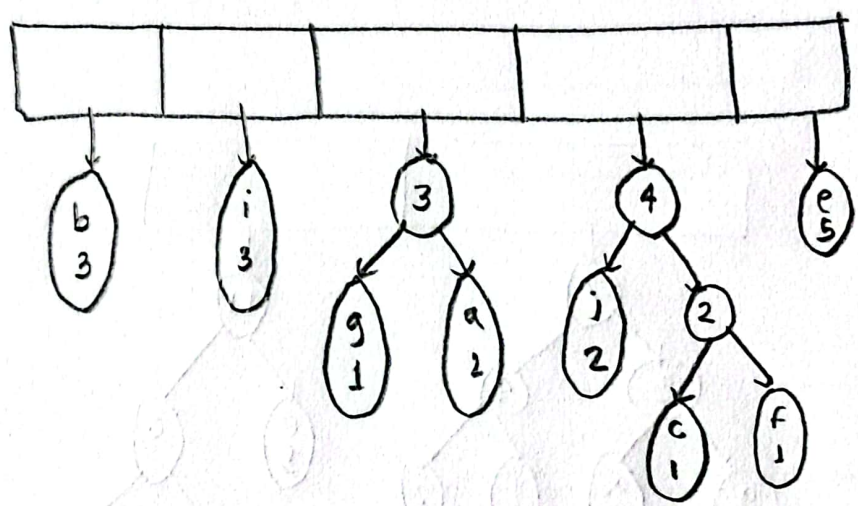
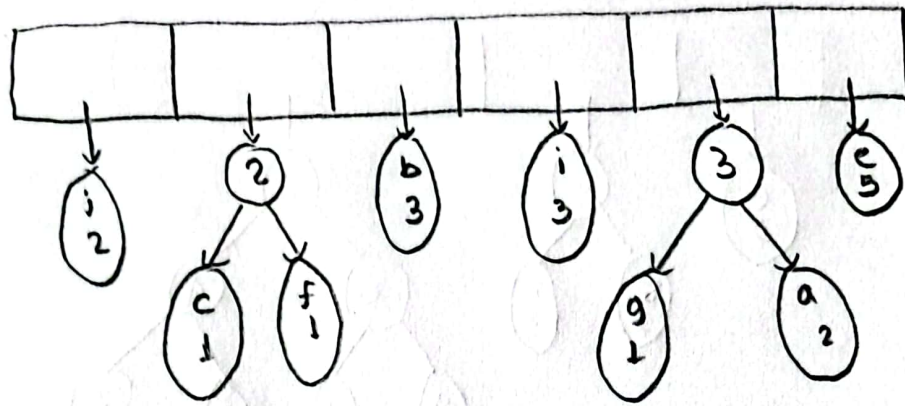
↪ a e e i e i e i b f b b g a e j c j

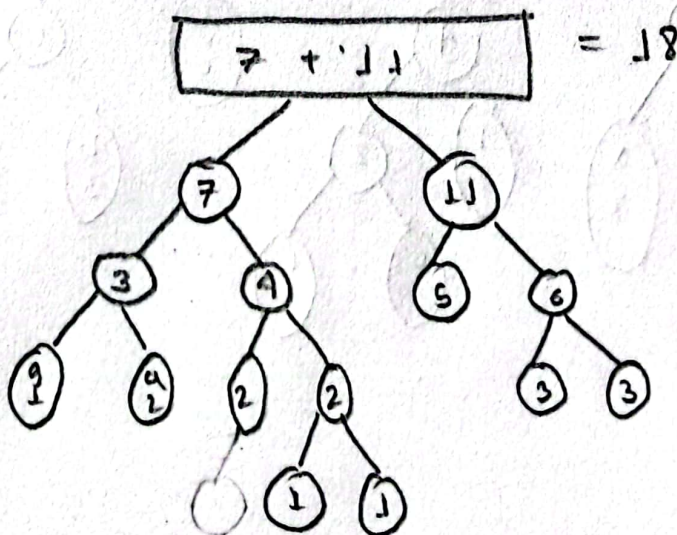
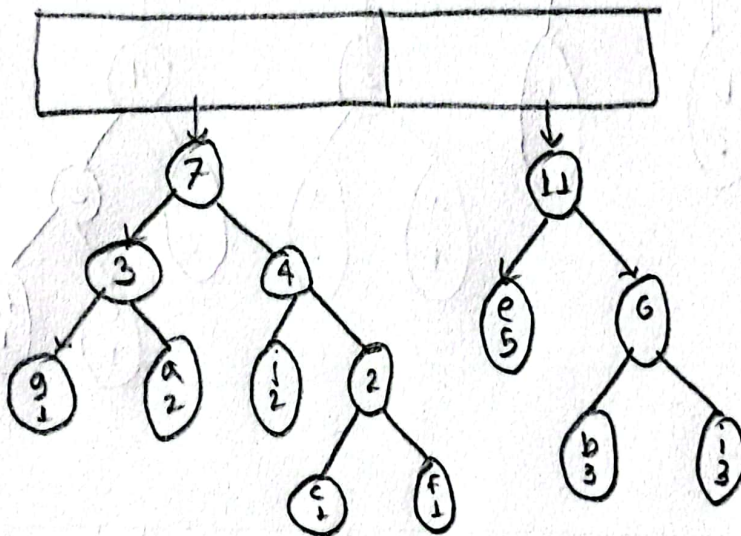
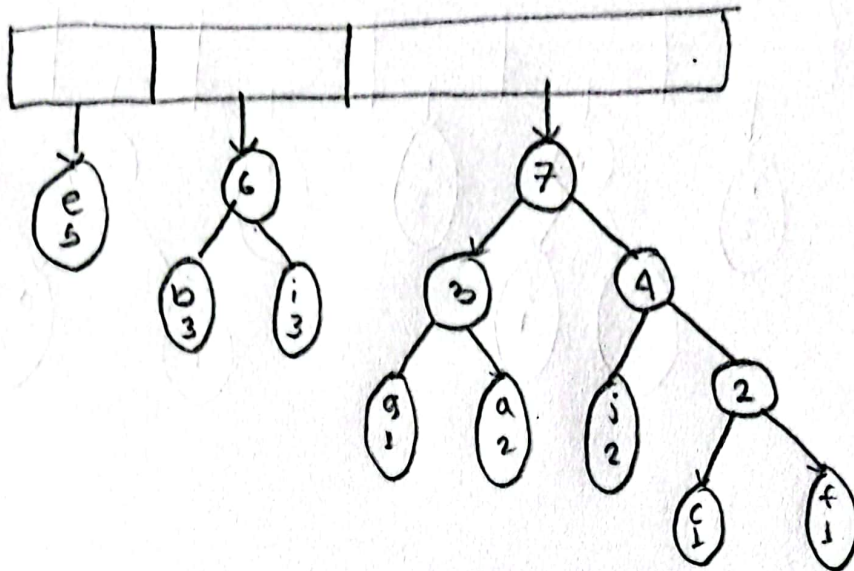
(b)

Character	Frequency	Character	Frequency
a	2	g	1
e	5	j	2
i	3	c	1
b	3		
f	1		

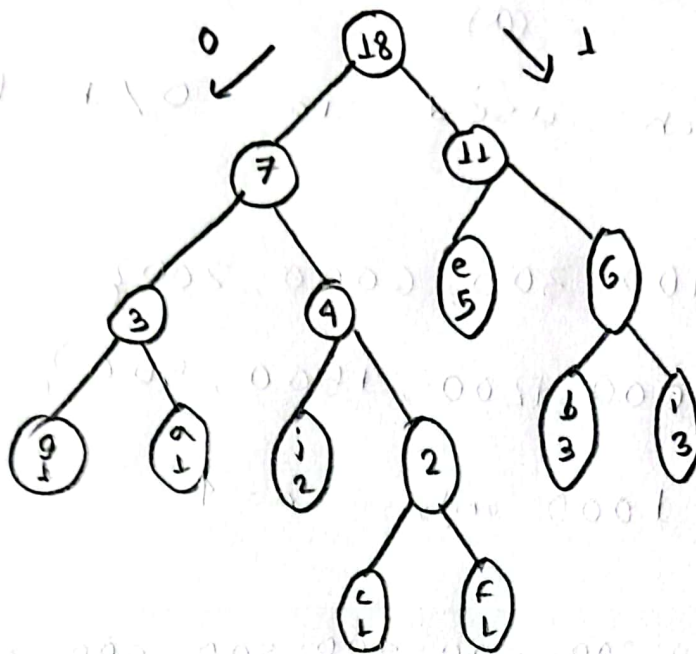
Tree [Arranging in Ascending Order]







Tree



a → 001

e → 10

i → 111

b → 110

f → 0111

j → 010

c → 0110

g → 000

a e e i e i e i b f b b g a e j c j

Encoded: 0011010111101111

01111100111110110

000001100100110

010

(c) The number of bits required is 51 bits

Answer to the 'Question No. 4

(a)

The approach used is 0/1 Knapsack

$$W = \{ 400, 300, 600, 200 \}$$

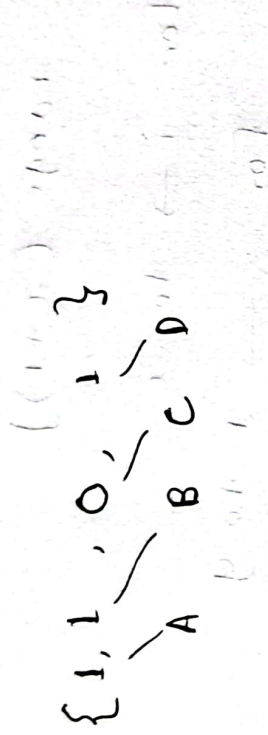
$$V = \{ 1000, 1200, 1500, 500 \}$$

$$W_{max} = 1000 \text{ units}, \quad n = 4$$

		$\xrightarrow{w}$											
v	w		0	100	200	300	400	500	600	700	800	900	1000
value	weight	0	0	0	0	0	0	0	0	0	0	0	0
500	200	1	0	0	500	500	500	500	500	500	500	500	500
1200	300	2	0	0	500	1200	1200	1700	1700	1700	1700	1700	1700
1000	400	3	0	0	500	1200	1200	1700	1700	2200	2200	2700	2700
1500	600	4	0	0	500	1200	1200	1700	1700	2200	2200	2700	2700

(b)

Optimal selection :-



Total profit / value : 2700 units

(c)

Fractional knapsack would have been the approach to maximize the

total value.

Answer to the Question No. 5

(a)

LCS or Longest Similar Subsequence should be implemented.

$$x = \text{len}(A)$$

$$y = \text{len}(B)$$

A

$z = [0 \text{ for } j \text{ in range}(y+1)]$ i in

range(x+1))

for $i \rightarrow 0, x,$

for $j \rightarrow 1 \text{ to } y$

if $A[i] == B[j]$, or (a)

wordnet(A[i], B[j])

then ~~z~~ $z[i, j]$,

then $z[i, j] \rightarrow 1 + z[i-1, j-1]$

else: $z[i, j] \rightarrow \max(z[i-1, j],$

$z[i, j-1])$

return $z[x, y]$

(c)

Com plexity $\rightarrow$ is $\rightarrow$ be analyzed

(b) A similarity percentage from the longest similar sequence of words

$$\left. \begin{array}{l} x = \text{len}(A) \\ y = \text{len}(B) \end{array} \right\} \rightarrow \max \rightarrow P$$

$\text{LCS}(A, B)$

$$\text{Similarity percentage} = \frac{\text{LCS}(A, B)}{P} \times 100\%$$

(c)

Complexity is to be analyzed.

$$x = [0] *$$

$$2D \text{ Arr} = [0] * (x+y+1)$$

$$[0] * (y+1)$$

$$\therefore, 2D \text{ Arr} = (x+1)(y+1)$$

It would take $O(1)$ complexity.

$$= O(xy)$$

$$= O(xy)$$

Checking similarity $\rightarrow O(xy) \times \underline{1}$

For each entry $\rightarrow O(xy) \times 1 \times 1$
 $= O(xy)$

So, total

complexity will be $= O(xy) + O(xy)$
 $= O(xy)$

$$T(n) = O(xy)$$